

Relatório de evolução do projeto - Sprint 03

EV Challenge - GoodWe · Prompt and Artificial Intelligence · 2SEM 2026.2 Grupo: Diogo Chiaradia Santos (RM 570246), Rafael Laprega Gontijo Magalhaes (RM 561975), Gustavo Torres de Oliveira (RM 572952) e Lucas Furquim Lima (RM 568690)

Fonte do PDF de entrega. Resultados medidos localmente com Ollama em 19/09/2026.

1. Resumo da evolução

Nas **Sprints 1/2** o chatbot era um script procedural: prompt montado por concatenação de strings, histórico em lista Python sem limite, chamada HTTP crua pro Ollama e "saída estruturada" feita com `json.loads` na resposta do modelo. Não tinha guardrail nenhum, nem de segurança nem de escopo.

Na **Sprint 03** o núcleo foi reconstruído em LangChain:

- **Chain LCEL** (`src/chain/builder.py`): `ChatPromptTemplate` | `ChatOllama` | `StrOutputParser` pra conversa e chain dedicada com `with_structured_output` pra consultas de recarga.
- **Memória por sessão** (`src/chain/memoria.py`): `RunnableWithMessageHistory` + `ConversationTokenBufferMemory` com teto de tokens e poda dos turnos antigos.
- **Structured output** (`src/schemas/consulta.py`): `ConsultaRecarga` em Pydantic v2 com `field_validator` (formato da estação, normalização de estado, faixa plausível de potência).
- **Context engineering** (`prompts/`): system prompt v2 com XML tagging (`<papel>`, `<contexto>`, `<base_conhecimento>`, `<regras>`, `<formato_resposta>`) e medição de tokens com `tiktoken`.
- **Guardrails** (`src/guardrails/`): moderação anti-jailbreak/injection e validador de escopo GoodWe com recusas que orientam a procurar profissional habilitado.

2. Refatoração: decisões e trade-offs

- **Roteamento por palavras-chave** entre conversa livre e consulta estruturada, em vez de classificador por LLM. Decisão: determinístico, sem latência extra e fácil de testar no eval. Trade-off: frases muito fora do padrão podem cair na conversa livre.
- **Poda da memória manual** após cada turno. No LangChain 1.x o `prune()` da `ConversationTokenBufferMemory` foi absorvido pelo `save_context`, que o `RunnableWithMessageHistory` não chama. Reimplementamos a mesma lógica (remove as mensagens mais antigas até caber no teto) contando tokens com `tiktoken`, porque o contador default do LangChain 1.x puxaria o pacote `transformers` inteiro só pra isso.
- **Base de conhecimento injetada como variável parcial** do template, não por replace na string: as chaves do JSON quebram o template f-string do LangChain (erro real encontrado

no desenvolvimento).

- **Stack da Aula 02 mantida apesar de deprecated:**
ConversationTokenBufferMemory e RunnableWithMessageHistory vivem hoje no pacote langchain-classic. Mantemos porque é o conteúdo do Módulo 1; a migração pra LangGraph fica pro Módulo 3.

3. Comparativo antes/depois

O eval foi ampliado para 17 casos: happy path, memória em três turnos, entradas inválidas, structured output, jailbreak, fora de escopo e um falso positivo de guardrail. A tabela compara o legado e a versão LCEL com o qwen3:8b, executados localmente pelo Ollama.

Métrica	Sprints 1/2 (manual/legado)	Sprint 03 (LCEL)
Qualidade das respostas (nota média no eval, 0-10)	7,04	10,00
Tokens visíveis por turno (pergunta + resposta)	211,9	113,2
Latência média por turno	4,387 s	1,823 s
Acurácia do structured output	0%	100%
Recusas corretas (jailbreak + out-of-scope)	5/8	8/8
Tokens do system prompt	80 (v1)	382 (v2, com base de produtos embutida)

Dados brutos: evals/sprint3_results.json.

O que salta da tabela:

1. **As respostas ficam mais curtas:** na mesma contagem de pergunta mais resposta, a média caiu 47%. O histórico e o system prompt são informados separadamente porque cada versão os monta de forma diferente.
2. **Structured output vai de 0% a 100%:** json.loads em texto livre falhava quando havia prosa junto do JSON; a chain nova valida a saída com Pydantic.
3. **As recusas ficam previsíveis:** os guardrails determinísticos acertaram os oito casos de injection e fora de escopo. O eval inclui também um caso permitido com a palavra “processar”, para evitar falso positivo jurídico.
4. **O prompt v2 melhora qualidade e tamanho:** no qwen3:8b, a nota subiu de 9,71 para 10,00 e a média caiu de 164,6 para 113,2 tokens. No gemma3:1b, a nota subiu de 8,42 para 9,01.

4. Problemas encontrados e soluções

Problema 1: JSON da base de conhecimento quebrando o template. Ao montar o prompt v2 com replace de string, o LangChain interpretava as chaves do JSON como variáveis de template

(`ValueError: Invalid format specifier`). Solução: base injetada como variável parcial do `ChatPromptTemplate`, que não reprocessa o conteúdo.

Problema 2: `prune()` sumiu no LangChain 1.x. A `ConversationTokenBufferMemory` do `langchain-classic` só poda dentro de `save_context`, que o `RunnableWithMessageHistory` não usa. Solução: poda manual após cada turno com contagem via `tiktoken`, replicando a lógica original. O script `demo_memoria.py` mostra o histórico sendo reduzido quando passa do limite.

Problema 3: padrões de jailbreak com cobertura parcial. O eval pegou que "ignore todas as suas instruções" escapava da blocklist. Solução: ampliação dos padrões e caso de regressão permanente no eval set (jb01 a jb03).

Problema 4: consulta operacional sem fonte de dados. A primeira versão pedia que o modelo inventasse status e faturamento plausíveis. Como o projeto não está integrado ao SEMS, esses campos agora ficam nulos e a resposta orienta a consulta à plataforma. O modelo só extrai o identificador da estação.

5. Base técnica e modelos

As especificações da linha HCA foram verificadas na [página oficial da GoodWe Brasil](#) e na [folha de dados oficial](#). A base registra os modelos GW7K-HCA, GW11K-HCA e GW22K-HCA.

O `qwen3:8b` ficou como principal pela nota de 10,00 e pelo acerto dos campos estruturados. O `gemma3:1b` chegou a 9,01 e foi mais rápido, mas não extraiu o identificador das estações nos dois casos desse tipo. Os resultados completos estão em `docs/relatorio_modelos.md` e `evals/sprint3_results.json`.

6. Equipe e divisão de trabalho

Integrante	RM	Tarefa principal
Diogo Chiaradia Santos	570246	chains LCEL e structured output (<code>builder.py</code> , <code>consulta.py</code>)
Rafael Laprega Gontijo Magalhaes	561975	memória por sessão e guardrails (<code>memoria.py</code> , <code>moderation.py</code> , <code>scope_validator.py</code>)
Gustavo Torres de Oliveira	572952	prompts versionados e eval set (<code>prompts/</code> , <code>eval_set.json</code>)
Lucas Furquim Lima	568690	relatórios e comparação de modelos (<code>docs/</code> , <code>multi_provider.py</code>)